Diseño de un pipeline de datos

Reglas, etapas y pruebas, con apoyo de agentes

Comercial Horizonte – Semana 2 – Clase 3

Prof. Mag. Ing. Mat. Diego Fernández

Diploma en Ciencia de Datos e Inteligencia Artificial Aplicada

90 minutos. Caso, fechas y UM ficticios.

Del diagnóstico de datos al proceso en Python

Semana 1: comprender los datos

Revisamos las ventas del caso y decidimos qué registros aceptar o excluir. Ese trabajo forma la **Entrega 1 (E1)**.

Semana 2: aplicar las mismas reglas con un programa

Diseñaremos un proceso que lea, valide, calcule y guarde resultados. La **Entrega 2 (E2)** conserva E1 y agrega el programa, sus resultados y las pruebas.

Pregunta guía: ¿cómo repetir el análisis y comprobar que está bien?

El caso: ¿qué queremos saber de las ventas?

Comercial Horizonte: empresa ficticia

Tenemos un archivo con ventas de tres categorías: **Oficina, Tecnologia y Hogar**. Cada fila de datos describe una venta.

- Queremos calcular cuánto se vendió, cuánto costó y qué margen quedó.
- Antes debemos revisar los datos: hay una repetición, una cantidad negativa y un precio faltante.
- Los importes se expresan en **UM: unidades monetarias ficticias**.

Datos de entrada → revisión → cálculos → resultados.

Primero: localizar y abrir el archivo de ventas

El archivo de partida se llama `ventas_original.csv`

Es la tabla original de ventas que utilizaremos como punto de partida. Es el mismo caso de la Semana 1; la conservaremos sin cambios.

1. En Prism: **Módulo 6** → `06_Recursos_del_Caso` → `ventas_original.csv`.
2. En el ZIP de recursos: descomprimir y abrir `Recursos_Semana_2/ventas_original.csv`.
3. Mirar el encabezado y la primera fila en un editor de texto o importar como CSV en una hoja de cálculo. **No guardar cambios sobre el original**.

Hay 16 filas de datos y 6 columnas. El encabezado no cuenta como venta.

Cómo leer el CSV: del encabezado a V001

Un CSV es una tabla guardada como texto; aquí la coma separa los campos. La primera línea nombra las columnas. Debajo aparece:

Primera fila de datos, copiada del archivo

```
V001,2026-10-05,Oficina,10,25,15
```

Columna	Valor de V001	Significado
`id`	V001	Identificador de esta venta.
`fecha`	2026-10-05	Fecha simulada de la venta.
`categoria`	Oficina	Grupo del artículo vendido.
`cantidad`	10	Unidades vendidas.
`precio_unitario`	25	Ingreso por cada unidad, en UM.
`costo_unitario`	15	Costo por cada unidad, en UM.

Los valores 10, 25 y 15 se leen del archivo; todavía no son cálculos.

V001 es un identificador; registro 1 es una posición

Registro: posición de la fila de datos, contando desde 1 y sin el encabezado. **ID**: código escrito en la columna `id`.

Registro	ID	Qué encontramos en `ventas_original.csv`
1	V001	Primera venta: 10 unidades, precio 25, costo 15.
5	V005	Primera aparición de esta venta.
14	V005	Repite exactamente los seis valores del registro 5.
15	V014	Cantidad -2: no cumple la regla del caso.
16	V015	Precio unitario vacío: no podemos calcular la venta.

Una distinción importante

El registro 15 tiene ID V014. No hay que buscar el ID V015 para localizarlo. Si copiamos una fila a otro archivo, su posición puede cambiar; su ID se conserva.

Qué aprenderemos a hacer

1. Diseñar las etapas: leer, validar, calcular y guardar.
2. Relacionar una regla con la parte del código que la aplica.
3. Anticipar resultados y distinguir excluir de detener.
4. Verificar productos y repetir una versión conservada.

Pausa de reflexión

Al terminar, tendrás que explicar una decisión con tus palabras.

Dónde encaja en Ingeniería de Datos y MLOps

Lo que practicamos ahora

Preparación de datos, pruebas, trazabilidad y repetición de un pipeline determinista: mismas entradas y reglas, mismos resultados de negocio.

- Son bases operativas útiles para MLOps.
- Un ciclo completo de ML también requiere evaluar modelos, desplegarlos y seguir su comportamiento.
- Esta semana no entrena ni despliega un modelo predictivo.

Después: agente analista, versiones, entorno reproducible y gestión de fallos.

Ruta de 90 minutos

Minutos	Trabajo
0–15	Abrir el caso: archivo, columnas, identificadores y entregas.
15–25	Entender el proceso y los documentos con sus reglas.
25–38	Leer y validar; reconocer decisiones en código.
38–50	Seguir V001 y V002 desde el archivo hasta sus cálculos.
50–62	Entender las políticas y los ejemplos de alerta.
62–75	Diseñar pruebas pequeñas y reconocer una detención.
75–83	Conservar el programa, repetirlo y dar el encargo al agente.
83–90	Respuesta individual y preparación de la Clase 4.

De resolver una vez a tener un procedimiento

Análisis puntual	Pipeline reutilizable
Responde sobre un archivo.	Recibe archivos con estructura acordada.
Puede dejar pasos sin registrar.	Explicita reglas y parámetros.
Entrega un resultado.	Conserva productos y decisiones.
Para repetir, puede requerir rehacer trabajo.	Ejecuta el mismo script guardado.

Script: archivo de instrucciones. Pipeline: el proceso organizado que esas instrucciones implementan.

Los documentos que explican cómo trabajar

Están junto al CSV en `06_Recursos_del_Caso`; también están en el ZIP entregado.

Archivo	Para qué lo consultamos
`diccionario_y_reglas.md`	Explica columnas y decisiones de calidad.
`contrato_pipeline.md`	Define entradas, pasos y productos esperados. Aquí «contrato» significa especificación de la tarea.
`politica_margen_v1.md`	Indican cuándo corresponde una alerta por margen bajo.
`politica_margen_v2.md`	
`matriz_8_pruebas.csv`	Organiza las comprobaciones P1 a P8.

Los archivos .md son documentos de texto que se pueden abrir y leer.

Antes de pedir código: acordar la tarea

- **Entrada:** `ventas_original.csv`, leído sin modificarlo.
- **Reglas:** consultar el diccionario, el contrato y las políticas.
- **Parámetros:** indicar la fecha simulada y una carpeta nueva para resultados.
- **Productos:** filas válidas, reporte de calidad y resumen de importes.
- **Comprobación:** anticipar resultados y contrastarlos con lo que produce el programa.

Pausa de reflexión

¿Qué quedaría sin decidir si pedimos solamente «limpia el archivo»?

Cuatro etapas, cuatro preguntas

Etapa	Pregunta	Evidencia
Leer	¿Es la entrada acordada?	Encabezado y origen.
Validar	¿Qué filas cumplen?	Decisiones y motivos.
Calcular	¿Qué importes producen?	Filas y agregados.
Guardar	¿Dónde queda el resultado?	Tres archivos nuevos.

Cada etapa debe permitir comprobar algo concreto.

Leer: comprobar la estructura del archivo

- **Esquema** significa los nombres, el orden y la cantidad de columnas acordadas: las seis que acabamos de leer.
- `ventas_original.csv` tiene 16 registros. Ese conteo identifica el archivo completo del caso.
- Para comprobar una regla podemos preparar un CSV pequeño, con el mismo encabezado y una o dos filas.
- El programa debe aceptar esos tamaños: no exigir siempre 16 filas.

Ejemplo de archivo de prueba

`Pruebas/sin_validas.csv` contiene los registros originales 15 y 16 (V014 y V015). Dentro de ese archivo ocupan las posiciones 1 y 2.

Validar: aceptar, excluir o detener

Aplicamos `diccionario_y_reglas.md`; no inventamos correcciones.

Situación	Decisión
V001 cumple las reglas.	Aceptar: incluirla en los cálculos.
Registro 14 repite V005 del registro 5.	Excluir la repetición; conservar la primera.
Registro 15: V014 tiene cantidad -2.	Excluir; no convertirla en $+2$.
Registro 16: V015 no tiene precio.	Excluir; no completar con un valor supuesto.
Ejemplo de fallo: columnas cambiadas o mismo ID con datos diferentes.	Detener todo el proceso y pedir revisión.

Excluir conserva la fila y el motivo en el reporte; evita que participe del cálculo. También se validan ID, fecha, categoría y valores numéricos según el diccionario.

Leer código: condición y detención

Fragmentos para reconocer decisiones; no son el programa completo.

```
if encabezado != columnas_esperadas:
    raise ValueError("Esquema incompatible")

motivos = []
if fila["precio_unitario"] == "":
    motivos.append("precio vacio")
```

- `if`: «si ocurre esta condición».
- `raise`: señala un fallo que debe detener el flujo.
- `append`: agrega un motivo; después se registran todos y se excluye la fila una sola vez.

Pausa de reflexión

¿Cuál de estas condiciones afecta todo el archivo?

V001: de los datos de la fila a los resultados

Fuente: primer registro de `ventas_original.csv`, que ya localizamos.

Datos leídos: 10 unidades; precio 25; costo unitario 15

$$\text{Ventas} = 10 \times 25 = 250 \text{ UM}$$
$$\text{Costos} = 10 \times 15 = 150 \text{ UM}$$
$$\text{Margen} = 250 - 150 = 100 \text{ UM}$$

Interpretación: ingresan 250 UM y el costo registrado es 150 UM; la diferencia es 100 UM. Aquí no se descuentan otros gastos.

Margen porcentual de V001: $100 \times 100/250 = 40\,\%$.

V002: segunda fila, y luego sumamos ambas ventas

Fuente: segundo registro de `ventas_original.csv`:
`V002,2026-10-05,Tecnologia,2,600,420`
2 unidades: ventas $2 \times 600 = 1200$; costos $2 \times 420 = 840$; margen 360 UM.

ID de venta	Unidades	Ventas (UM)	Costos (UM)	Margen (UM)
V001	10	250	150	100
V002	2	1200	840	360
Solo estas dos	12	1450	990	460

Porcentaje de estas dos ventas, no de todo el archivo

$100 \times 460/1450 = 31{,}724137\ldots$ %. Se muestra 31,72 %.

V001: $100/250 = 40$ %; V002: $360/1200 = 30$ %. Promediarlos da 35 %, aunque las ventas tienen distinto tamaño: **primero sumamos importes y después dividimos**.

Leer código: sumar y guardar

Fragmento ilustrativo: las filas ya fueron validadas y sus importes calculados. **Decimal** es una herramienta de Python para operar con números decimales.

```
ventas = sum((f["ventas"] for f in validas), Decimal("0"))
costos = sum((f["costos"] for f in validas), Decimal("0"))
margen = ventas - costos
resumen = {"ventas": str(ventas), "margen": str(margen)}
with open(destino / "resumen.json", "x", encoding="utf-8") as f:
    json.dump(resumen, f, ensure_ascii=False, indent=2)
```

- `sum`: suma los importes de las filas válidas.
- `str`: conserva el decimal como texto en JSON.
- `json.dump`: escribe; modo `x` evita sobrescribir.

Fragmento parcial: faltan categorías, política, porcentaje y otros productos.

Guardar: cada archivo responde una pregunta

Estos son **archivos de salida que creará el programa**. JSON guarda datos con nombres y valores, por ejemplo `{"ventas": "250"}`.

- `ventas_validas.csv`: ¿qué filas entraron al cálculo?
- `reporte_calidad.json`: ¿qué se excluyó y por qué?
- `resumen.json`: ¿cuáles son los agregados y qué política se aplicó?

Trazabilidad

Poder volver del resultado a la fuente, la regla y la ejecución que lo produjo. Guardar código, parámetros y comando real en la bitácora.

Ahora usamos un ejemplo inventado para entender la alerta

Ejemplo hipotético; no es una fila del CSV original

Suponemos ventas de 100 UM y costos de 73 UM. El margen es $100 - 73 = 27$ UM y su porcentaje es $100 \times 27/100 = 27\,\%$.

Reglas tomadas de los archivos de política del caso: v1 y v2 son dos versiones de la regla de alerta.

Fecha simulada	Regla aplicable	Resultado
18/10/2026	v1: alerta si el margen es menor que 25 %	SIN ALERTA
19/10/2026	v2: alerta si el margen es menor que 30 %	ALERTA

Cambia la regla; los importes de este ejemplo permanecen iguales.

Qué fecha selecciona la política

- v1: 05/10/2026 a 18/10/2026, ambos incluidos.
- v2: desde 19/10/2026, sin cierre declarado.
- Se usa la **fecha simulada de ejecución**, no la fecha de cada venta ni el día real de clase.
- Debe existir exactamente una política válida. Si falta o se solapa, detener y pedir aclaración.

Pausa de reflexión

¿Qué datos conservarías para explicar por qué cambió una alerta?

Dos datos de prueba para entender el redondeo

Son **ejemplos preparados para probar la regla**, separados del CSV original. En ambos: 1 unidad, precio 100 y fecha simulada 19/10/2026 (v2).

Costo unitario	Margen (%)	Mostrado	Estado
70	30 %	30,00 %	SIN ALERTA
70,001	29,999 %	30,00 %	ALERTA

Archivos: `Pruebas/frontera_30.csv` y `Pruebas/redondeo_29999.csv`, incluidos en el ZIP. En el CSV se usa punto decimal: `70.001`.

- Se alerta si el porcentaje es **menor** que 30; la igualdad no alerta.
- Se compara el valor calculado antes de redondear para mostrarlo.
- Sin filas válidas: porcentaje `null` (no definido) y estado NO EVALUABLE, con política válida.

Una prueba empieza antes de ejecutar

1. Elegir una entrada pequeña y una regla.
2. Escribir el resultado esperado de forma independiente.
3. Ejecutar y abrir los productos.
4. Comparar esperado y observado; conservar evidencia.

Cómo preparar la prueba con V001

Crear un CSV nuevo con el encabezado original y solo su primera fila de datos. Esperamos una fila válida, ventas 250, costos 150 y margen 100. No se borra nada del original.

P1 a P8: nombres de las ocho pruebas

P significa prueba. Sus consignas se encuentran en `matriz_8_pruebas.csv`; allí se registran también el resultado observado y la evidencia.

Pruebas	Qué comprobar
P1–P4	Archivo original completo: conteos, V005 una sola vez, exclusión de V014 y V015 con sus motivos.
P5–P6	Sus resultados: importes por categoría y total; porcentaje calculado después de sumar.
P7	Mismo original con fechas simuladas 18 y 19 de octubre; archivos separados para el límite de 30 % y el redondeo.
P8	Archivo `sin_validas.csv`: V014 y V015; dos leídas, cero válidas, dos excluidas.

Una ejecución puede aportar evidencia a varias pruebas.

Demostrar una detención también enseña

1. Crear una copia de prueba del CSV.
2. Cambiar solo el encabezado `precio_unitario` por `precio`.
3. Predecir: mensaje de esquema incompatible y señal de fallo.
4. Ejecutar hacia una carpeta nueva; no presentar resumen de éxito.

Pausa de reflexión

Si la carpeta de ayer tiene un resumen, ¿prueba que la ejecución de hoy funcionó?

D1 significa demostración 1. Se registra en la bitácora (registro de lo realizado) y complementa las ocho pruebas.

Repetición 1 (R1): ejecutar el mismo programa

R1 es el nombre de esta comprobación de repetición.

1. Guardar la versión revisada de `pipeline.py`.
2. Repetir con mismos datos, políticas, fecha e intérprete.
3. Usar otra carpeta de salida.
4. Comparar filas, exclusiones, importes, porcentaje y estado.

Qué puede cambiar

Ruta, hora o identificador de ejecución. Documentar esos metadatos; los resultados de negocio deben coincidir.

Repetir un resultado no demuestra que sea correcto: también se necesitan las pruebas.

Encargo al agente: indicar archivos y pedir un plan

Adjuntar `ventas_original.csv`, `diccionario_y_reglas.md`, `contrato_pipeline.md`, ambas políticas y la matriz de pruebas.

Prompt para iniciar

Trabaja con estos archivos de Comercial Horizonte. Primero identifica la entrada, explica sus seis columnas y muestra de dónde salen los valores de V001. Propón los pasos leer, validar, calcular y guardar. Después genera un script legible en Python usando biblioteca estándar y Decimal. Aplica las reglas de los documentos sin modificar el original. Indica fecha simulada y carpeta nueva de salida. Explica cómo ejecutarlo y comprobar sus tres productos. Distingue lo ejecutado de lo pendiente; si no puedes abrir un archivo, indícalo.

El estudiante revisa el plan antes de ejecutar y contrasta los resultados. Los prompts completos están en el libro y la guía práctica.

Si falla una prueba, corregir con evidencia

- Informar entrada, esperado y observado.
- Corregir implementación; conservar reglas y fuente.
- Guardar una nueva versión.
- Repetir la prueba fallida y luego P1–P8: regresión.
- Si no hubo fallo, justificar la aceptación provisional.

Pausa de reflexión

En la prueba que usa solo V001 y V002, el programa informa 35 %. ¿Qué parte del cálculo revisarías frente al 31,72 % esperado?

Respuesta individual por el chat

1. «La cantidad negativa se _____ porque...».
2. «El esquema cambiado hace que el proceso _____».
3. «Un archivo de dos filas es admisible si...».
4. «Para repetir sin regenerar, conservaré...».

Cada estudiante explica una regla y localiza dónde se aplica. Un segundo agente ayuda, pero no garantiza revisión independiente.

Qué preparar para la Clase 4

- Abrir los recursos: CSV original, diccionario, contrato, políticas y matriz de pruebas.
- Conservar la **Entrega 1 (E1)**: diagnóstico y comprobaciones de la Semana 1.
- Llevar los cálculos manuales de V001 y V002; serán nuestra referencia para revisar el programa.
- La **Entrega 2 (E2)** agrega el script, tres archivos de resultados, ocho pruebas y la bitácora acumulativa.
- Registrar también la detención D1 y la repetición R1. Completar evidencias en las dos horas autónomas previstas.

Debemos poder explicar de dónde sale cada dato y cómo se obtiene cada resultado.